



SSC502 – Laboratório de ICC I

Estruturas

Prof.Dr. Danilo Spatti

- As variáveis vistas até agora podem ser classificados em **duas** categorias:
 - simples: definidas por tipos int, float, double e char.
 - compostas homogêneas (ou seja, do mesmo tipo): definidas por array.

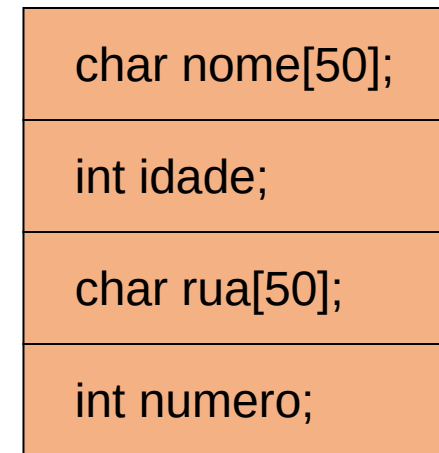
- No entanto, a linguagem C permite que se criem novas **estruturas** a partir dos tipos **básicos**.
 - struct

- Uma estrutura pode ser vista como um **novo** tipo de **dado**, que é formado por **composição** de variáveis de outros **tipos**.

```
struct nome_struct{  
    tipo1 campo1;  
    tipo2 campo2;  
    (...)  
    tipo campoN;  
};
```

- Tendo **informações** de uma **mesma pessoa**, tais informações podem ser **agrupadas**. Isso **facilita** também lidar com **dados** de outras **pessoas** no mesmo **programa**.

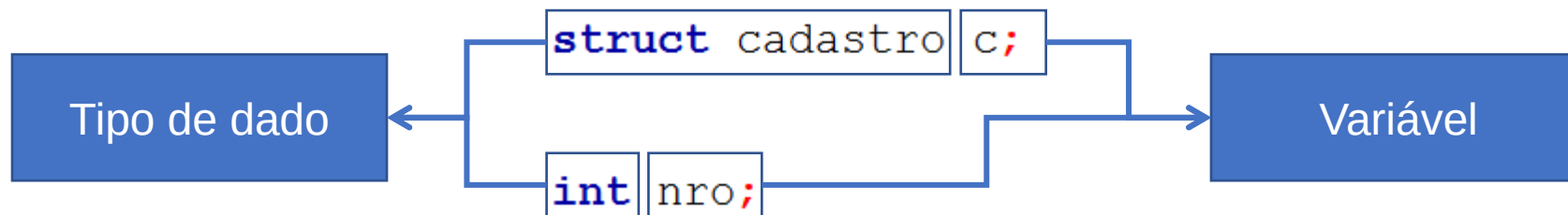
```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```



cadastro

- Uma vez **definida** a estrutura, uma **variável** pode ser **declarada** de modo similar aos **tipos** já **existentes**:

```
struct cadastro c;
```



- Por ser um tipo **definido** pelo **programador**, usa-se a palavra **struct** antes do **tipo** da nova **variável**.
- Exemplo: Declare uma estrutura capaz de armazenar o **número** e 3 **notas** para um **dado aluno**.

```
1  # include <stdio.h>
2
3  struct aluno{
4      int ra_aluno, nota1, nota2, nota3;
5  };
6
7  int main()
8  {
9      return 0;
10 }
```

- O uso de **estruturas facilita** na **manipulação** dos dados do **programa**. Imagine **declarar** 4 cadastros, para 4 pessoas diferentes:

```
char nome1[50], nome2[50], nome3[50], nome4[50];  
int idade1, idade2, idade3, idade4;  
char rua1[50], rua2[50], rua3[50], rua4[50];  
int numero1, numero2, numero3, numero4;
```


- Utilizando uma **estrutura**, o mesmo **pode** ser feito da **seguinte** maneira:

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

```
struct cadastro c1, c2, c3, c4;
```

- Cada variável da estrutura pode ser acessada com o operador ponto “.”

```
struct cadastro c;  
strcpy(c.nome, "João");  
scanf("%d", &c.idade);  
strcpy(c.rua, "Avenida 1");  
c.numero = 1082;
```

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

- Como nos **vetores**, uma estrutura pode ser **previamente inicializada**:

```
struct ponto{  
    int x;  
    int y;  
};  
struct ponto p1 = {10, 9};
```

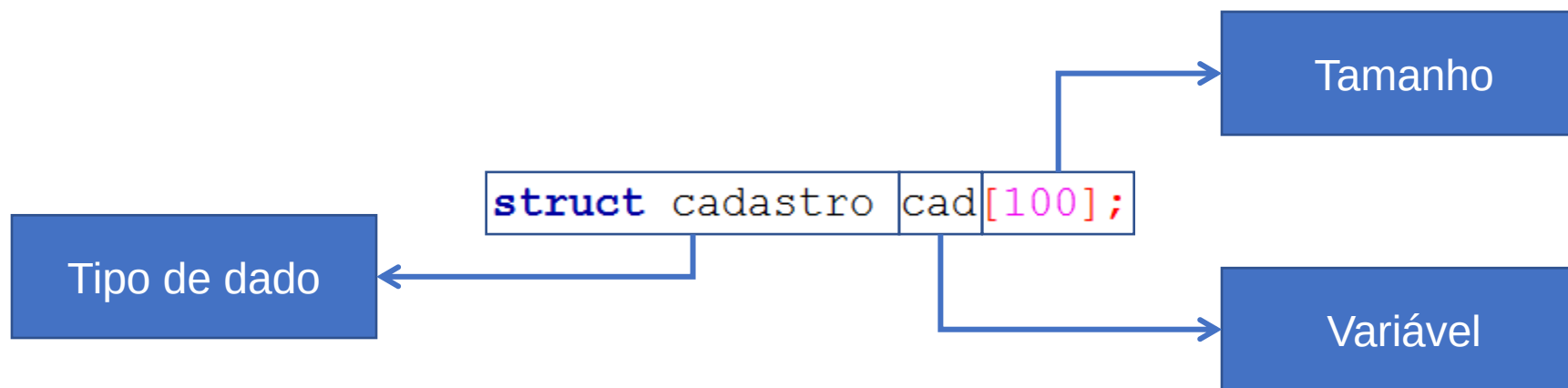
- Para leitura, deve-se ler cada variável, independentemente, respeitando seus tipos.

```
struct cadastro c;  
gets(c.nome);  
scanf("%d", &c.idade);  
gets(c.rua);  
scanf("%d", &c.numero);
```

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

- Se, ao invés de **5** cadastros, quisermos fazer **100** cadastros de pessoas?
- SOLUÇÃO: criar um **vetor** de **estruturas**.
- Sua **declaração** é **similar** a declaração de um **vetor** de um tipo **básico**.

- Desse modo, **declara-se** um **vetor** de 100 posições, onde **cada** posição é do tipo **struct cadastro**.



- **struct**: define um “conjunto” de variáveis que **podem** ser de tipos diferentes.
- **vetores**: é uma “lista” de **elementos** de **mesmo tipo**.

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

cad[0]

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

cad[1]

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

cad[2]

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

cad[3]

```
struct cadastro{  
    char nome[50];  
    int  idade;  
    char rua[50];  
    int  numero;  
};
```

- Num vetor de estruturas, o operador de **ponto** (.) vem **depois** dos **colchetes** ([]) do índice do **vetor**.
- Ex: fazer um cadastro de 4 pessoas, com os campos nome, idade, rua e número da rua.


```
1  # include <stdio.h>
2
3  struct cadastro{
4      char nome[50];
5      int  idade;
6      char rua[50];
7      int  numero;
8  };
9
10
11
12  int main()
13  {
14      struct cadastro c[4];
15      int i;
16      for (i = 0; i < 4; i++)
17      {
18          printf("\nDigite o nome %d: ",i+1);
19          gets(c[i].nome); setbuf(stdin, NULL);
20          printf("\nDigite a idade %d: ",i+1);
21          scanf("%d",&c[i].idade); setbuf(stdin, NULL);
22          printf("\nDigite a rua %d: ",i+1);
23          scanf("%[^\n]s",c[i].rua); setbuf(stdin, NULL);
24          printf("\nDigite o numero %d: ",i+1);
25          scanf(" %d",&c[i].numero); setbuf(stdin, NULL);
26      }
27      return 0;
28  }
```

```
Digite o nome 1: Danilo Hernane Spatti
Digite a idade 1: 37
Digite a rua 1: Av Sao Carlos
Digite o numero 1: 400
Digite o nome 2: Danilo Spatti
Digite a idade 2: 37
Digite a rua 2: Av Trabalhador Sancarlense
Digite o numero 2: 400
Digite o nome 3:
```

Lembrar de limpar
o buffer

- Utilizando a estrutura abaixo, faça um programa para ler o RA e três notas de 10 alunos, e armazene a média:

```
struct aluno{  
    int ra;  
    float nota1, nota2, nota3;  
    float media;  
};
```

```
1  # include <stdio.h>
2
3  struct aluno{
4      int ra;
5      float nota1, nota2, nota3;
6      float media;
7  };
8
9  int main()
10 {
11     struct aluno a[10];
12     int i;
13     for (i = 0; i < 10; i++)
14     {
15         printf("\n=====");
16         printf("\nDigite o RA e 3 notas separadas por espaco:\n");
17         scanf("%d %f %f %f",&a[i].ra,&a[i].nota1,&a[i].nota2,&a[i].nota3);
18         setbuf(stdin,NULL);
19         a[i].media = (a[i].nota1 + a[i].nota2 + a[i].nota3)/3.0;
20         printf("\nRA: %d Media: %f",a[i].ra,a[i].media);
21     }
22     return 0;
23 }
```

=====

Digite o RA e 3 notas separadas por espaco:
1230 5 6 7

RA: 1230 Media: 6.000000

=====

Digite o RA e 3 notas separadas por espaco:
1231 5.1 5.2 5.3

RA: 1231 Media: 5.200000

=====

Digite o RA e 3 notas separadas por espaco:
1232 5 6 9

RA: 1232 Media: 6.666667

=====

Digite o RA e 3 notas separadas por espaco:

- Atribuições entre **estruturas** só podem ser **feitas** quando as estruturas são **AS MESMAS**, ou seja, possuem o **mesmo nome**!

```
struct cadastro c1, c2;  
    c1 = c2; //correto
```

```
struct cadastro c1;  
struct ficha c2;  
C1 = c2; //incorreto
```

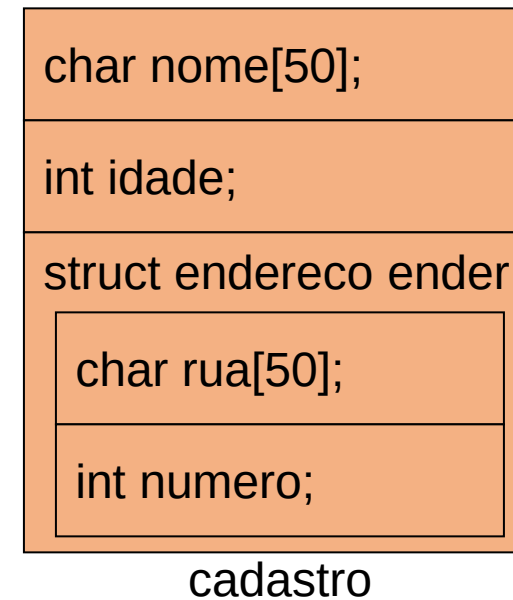
- No caso de **estarmos** trabalhando com vetores, a atribuição entre diferentes **elementos** do vetor é **válida**.

```
struct cadastro c[10];  
    c1 = c2; //correto
```

- Note que nesse caso, os tipos dos **diferentes** elementos do **vetor** são sempre **IGUAIS**.

- Sendo uma **estrutura** um tipo de **dado**, podemos **declarar** uma **estrutura** que utilize outra **estrutura** previamente **definida**:

```
struct endereco{  
    char rua[50];  
    int  numero;  
};  
struct cadastro{  
    char nome[50];  
    int  idade;  
    struct endereco ender;  
};
```



- O **acesso** aos dados do **endereço** do **cadastro** é feito **utilizando** novamente o operador **ponto** “.”.

```
struct cadastro c;  
gets(c.nome);  
scanf("%d", &c.idade);  
gets(c.ender.rua);  
scanf("%d", &c.ender.numero);  
strcpy(c.nome, "Joao");  
c.idade = 34;  
strcpy(c.ender.rua, "Avenida 1");  
c.ender.numero = 131;
```

```
struct endereco{  
    char rua[50];  
    int  numero;  
};  
struct cadastro{  
    char nome[50];  
    int  idade;  
    struct endereco ender;  
};
```

Lembrar de limpar
o buffer

```
struct ponto{  
    int x, y;  
};
```

```
struct retangulo{  
    struct ponto inicio, fim;  
};
```

```
struct retangulo = {{10,20},{30,40}};
```


- A linguagem C **permite** que o programador **defina** os seus **próprios** tipos com base em **outros tipos** de dados **existentes**.
- Para isso, utiliza-se o comando **typedef**, cuja forma geral é:

```
typedef tipo_existente novo_nome;
```

```
1  # include <stdio.h>
2
3  typedef int inteiro;
4
5  int main()
6  {
7      int x = 10;
8      inteiro y = 20;
9      y = y + x;
10     printf("Soma = %d\n", y);
11     return 0;
12 }
```

Soma = 30

Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

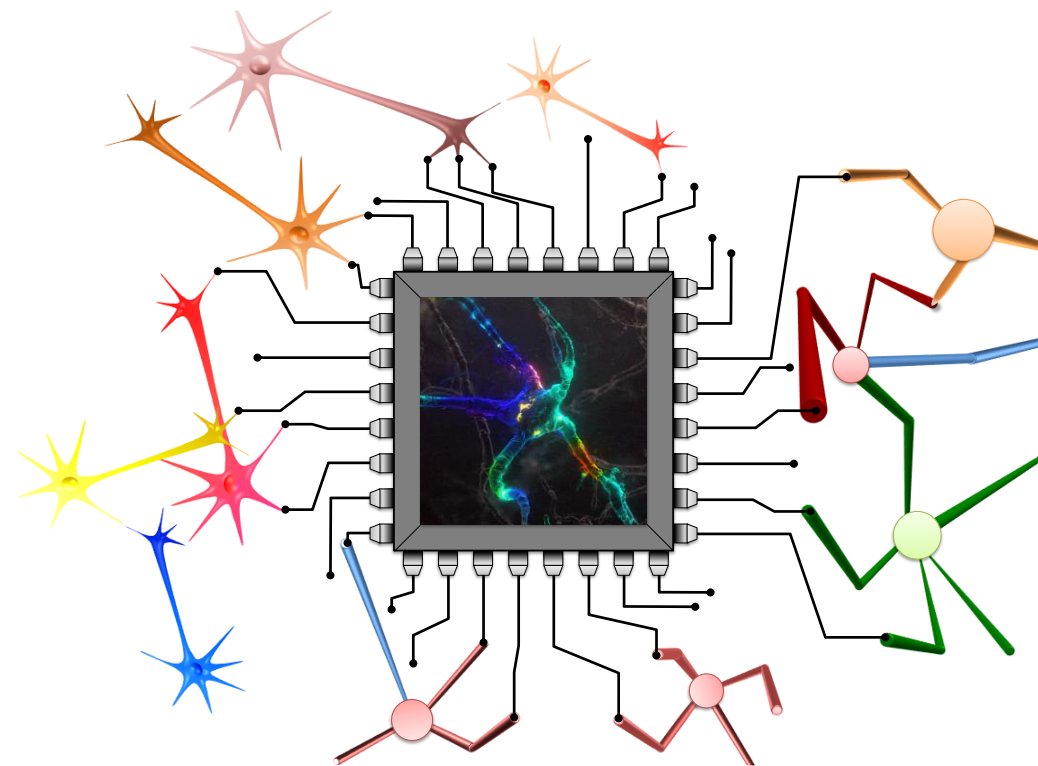
- O typedef é muito **utilizado** para definir **nomes** mais **simples** para **estrutura**, evitando **carregar** a palavra **struct** sempre que **referenciamos** a estrutura.

```
struct cadastro{
    char nome[300];
    int  idade;
};

typedef struct cadastro CadAlunos;

int main()
{
    struct cadastro aluno1;
    CadAlunos aluno2;
    return 0;
}
```

spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos